프로젝트 개요

김원준 | 신입 게임 클라이언트 프로그래머 기술 포트폴리오

3D Action Combat System

Zenless Zone Zero의 전투 흐름을 참고한 Unity 6 기반 3D 액션 개인 프로젝트입니다. 공격·회피·패링·교대가 이어지는 과정에서 데이터와 상태의 소유권을 일관되게 유지하는 데 초점을 맞췄습니다.



원본 프로젝트의 실제 파티 편성 화면. 신규 생성 이미지 아님.

개발 기간 / 형태	2026.05 - 2026.11 / 1인 개발
개발 환경 및 사용 기술	Unity 6000.3.13f1 · C# · URP 17.3.0 · Shader Graph Cinemachine 3.1.6 · Input System 1.19.0 · Timeline 1.8.12
구현 범위	전투·캐릭터 기믹·적 AI·교대·UI·씬 흐름과 연출 연동
외부 리소스	모델·애니메이션·사운드·VFX 등 외부 에셋 활용. 게임플레이 시스템 구현 및 에셋 연동 직접 수행.

핵심 플레이는 Maria·Erika·Eve의 파티 편성과 교대 전투입니다. 서로 다른 전투 기믹을 가진 세 캐릭터를 상황에 맞게 교대하며, 회피·패링·지원 공격을 연계하는 액션 게임입니다.

공개 소스 저장소 / README
플레이 영상 링크
실행 파일 링크

목차와 핵심 시스템

시스템별 책임 분리와 데이터 흐름, 상태 관리 및 확장 구조를 중심으로 구성

목차

01 프로젝트 개요
02 목차와 핵심 시스템
03 전투 시스템 구조와 처리 흐름
04-17 핵심 기술 구현
18 교대 퇴장 중 이동 오류 해결
19 궁극기 프리팹 사전 풀링
20 핵심 역량과 향후 발전 방향

핵심 시스템

핵심 시스템 / 페이지	주요 설계 및 구현 내용
01 공통 전투 판정 및 피해 처리 04p ~ 05p	판정 방식이 달라도 피해 계약을 유지한다. 공격 판정과 피해 처리의 책임을 분리해 근접·투사체·설치형 공격을 공통 흐름으로 처리한다.
02 교대와 특수 교대 06p ~ 09p	일반 교대·패링 지원·빠른 지원·콤보스킬의 발동 조건과 실행 흐름, 스크립트별 책임을 설명한다.
03 캐릭터 기믹 확장 10p ~ 13p	Hero FSM과 IHeroMechanic의 책임을 분리하고, Maria의 강화·Erika의 자동 회피와 여진 공격을 전용 기믹으로 구현한다.
04 적 AI 14p ~ 17p	Sensor·Blackboard·Decision·Pattern으로 판단 정보를 구성하고, BT의 행동 선택을 Enemy FSM의 상태 실행으로 연결한다.

공개 소스 저장소 / README
플레이 영상 링크
실행 파일 링크

전투 시스템 구조와 처리 흐름

입력·AI의 행동 요청부터 공격 판정, 피해 처리, 결과 반영까지의 전투 흐름

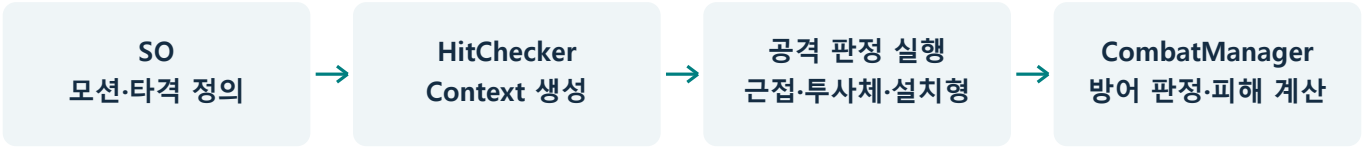


스크립트	역할 및 책임
Hero / IHeroMechanic	Hero는 공통 행동과 FSM 상태 전환을 관리한다. IHeroMechanic은 캐릭터별 전투 기믹을 공통 Hero 로직에 연결하는 인터페이스를 정의한다.
PartyStateManager / PartySwapManager	PartyStateManager는 파티 구성원과 현재 조작 캐릭터를 관리한다. PartySwapManager는 교대 요청 시 일반 교대·패링 지원·빠른 지원 여부를 판단하고, 교대할 캐릭터의 등장 위치와 진입 상태를 결정해 교대를 실행한다.
CombatDataSO	공격별 피해·그로기 계수, 타격 시점, 판정 방식 등의 설정을 저장한다. 실행 로직과 공격 데이터를 분리해 코드 수정 없이 공격 설정을 조정할 수 있도록 한다.
CombatEventContext / HitInfo	CombatEventContext는 공격 주체와 공격 계수를 담고, 판정 시 확인한 대상·충돌 위치·방향을 피해 처리 단계에 전달한다. HitInfo는 계산된 피해량과 피격 반응 정보를 담아, 대상이 피해를 처리할 수 있도록 전달한다.
CombatManager / IDamageable. TakeDamage(HitInfo)	CombatManager는 방어 판정과 피해량 계산을 담당한다. Hero·Enemy는 IDamageable 인터페이스의 TakeDamage를 구현해 실제 피해 적용과 피격·사망 처리를 수행한다.
Presentation / Pool / UI	Presentation은 전투 상황에 따른 연출을 처리하고, Pool은 반복 사용하는 객체를 재사용하도록 관리한다. UI는 체력·전투 자원 등 현재 상태를 화면에 표시한다.

Interfaces.cs
HeroCombatDataSO.cs
CombatEventArgs.cs

공통 전투 판정 및 피해 처리

01 근접·투사체·설치형 공격을 공통 전투 데이터와 피해 처리 흐름으로 연결



설계 배경

근접·투사체·설치형 공격은 판정 방식과 적중 시점이 다르지만, 방어 판정과 피해 계산에는 동일한 규칙이 적용되어야 합니다. 특히 투사체와 설치형 공격은 캐릭터가 다른 행동을 하거나 교대한 뒤에도 적중할 수 있습니다. 따라서 공격 생성 시 정해진 종류와 계수를 유지하면서, 각 판정 결과를 공통 피해 처리로 연결하는 구조가 필요했습니다.

해결 방식

HitChecker는 공격 데이터 SO에서 현재 공격의 설정을 읽고, 공격 종류·계수·판정 방식 등을 담은 CombatEventContext를 생성합니다. 이후 SO에서 제공하는 판정 방식에 따라 HitSystem의 근접·투사체·설치형 처리로 전달합니다.

근접 공격은 즉시 범위를 검사하고, 투사체와 설치형 공격은 전달받은 Context를 보관해 충돌하거나 예정된 타격 시점에 판정합니다. 이를 통해 공격을 시작한 캐릭터의 상태가 바뀌어도 생성 당시의 공격 정보를 유지합니다.

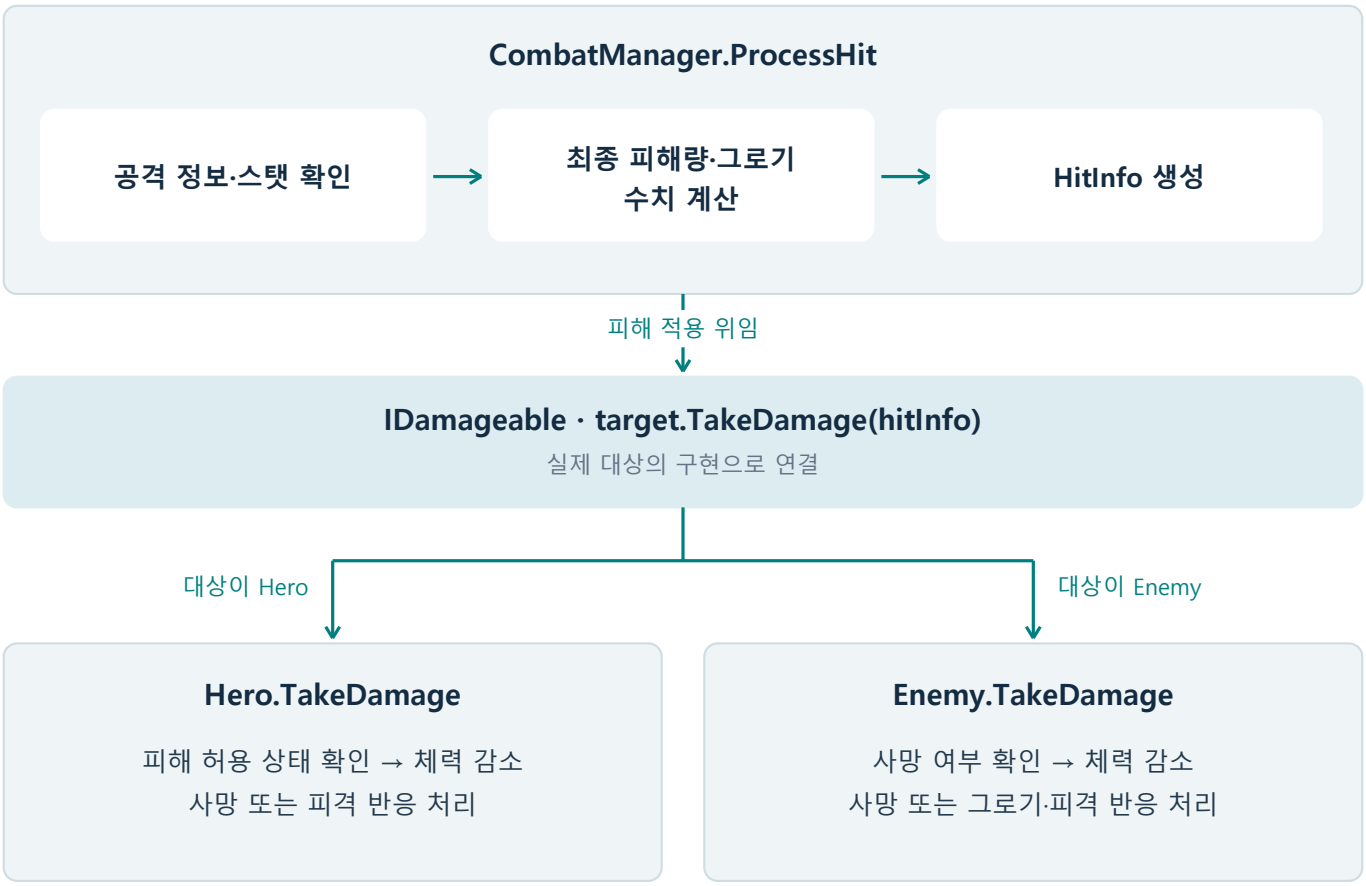
경로	생성 시점과 적중 시점
MeleeOverlap	타격 시점에 범위 내 대상을 검사하고, 대상별로 공통 피해 처리를 요청한다.
ProjectileCollision	발사 시 전달받은 Context를 보관하고, 충돌 시 단일 대상 또는 폭발 범위 내 대상을 처리한다.
PlacedArea	타격별 Context와 실행 시간표를 전달받아, 정해진 시점마다 범위 판정을 수행한다.

HeroCombatDataSO.cs
CombatEventArgs.cs
HeroHitChecker.cs

공통 전투 판정 및 피해 처리

02 CombatManager의 피해량 계산부터 Hero·Enemy의 TakeDamage를 통한 피해 적용까지

피해 처리 흐름



피해량 계산과 전달

CombatManager는 공격 정보와 공격자·대상의 스탯을 바탕으로 최종 피해량과 그로기 수치를 계산합니다. 계산 결과는 치명타 여부·피격 타입·반응 방향 등 피해 적용에 필요한 정보와 함께 HitInfo 구조체에 담습니다.

HitInfo는 대상별로 계산된 결과를 하나의 값으로 묶어 전달합니다. 처리 과정에서 상태를 공유할 필요가 없는 결과 데이터이므로, 값으로 복사해 전달하는 구조체로 정의했습니다.

Hero·Enemy의 피해 적용

Hero와 Enemy는 공격 판정 방식이나 계산 과정을 다시 확인할 필요 없이, 전달받은 HitInfo를 각자의 TakeDamage에서 적용합니다.
이를 통해 공통 피해 계산과 캐릭터별 피해 적용의 책임을 분리합니다.

CombatManager.cs
Hero.cs
Enemy.cs

교대와 특수 교대

01 일반 교대 — 파티 상태 조회부터 조작 Hero 전환까지

교대 시스템의 역할 분담

스크립트	역할 및 책임
PartyStateManager	파티 구성·현재 Hero를 보관하고 교대 후보를 조회한다.
PartySwapManager	요청을 검증하고 퇴장·등장·입력·카메라 전환을 실행한다.
HeroSwapSystem	개별 Hero의 교대 가능 상태, 잔여 행동과 표시·물리를 처리한다.
QuickAssistManager	빠른 지원 창과 실행에 필요한 Hero·대상 정보를 관리한다.
ChainAttackManager	콤보스킬의 후보·선택·실행 단계와 사용한 Hero를 관리한다.

일반 교대의 실행 순서

01 PartySwapManager

TryRequestSwap
빠른 지원 창을 먼저 확인한 뒤 일반·패링 교대 타입과 요청 가능 상태를 검사한다.

↓

02 PartyStateManager

GetValidSwapHero
입력 방향에서 살아 있고, 오프필드 행동 중이 아닌 InActive Hero를 찾는다.

↓

03 PartySwapManager → HeroSwapSystem

ExecuteSwap / BeginSwapOutProcess
일반 교대 위치의 충돌을 먼저 검사하고, 이전 Hero의 입력을 끈 뒤 퇴장을 시작한다.

↓

04 PartySwapManager → PartyStateManager

등장 준비 → SwapIn → TrySetCurrentHero
새 Hero의 표시·물리·입력·카메라를 준비한 뒤 현재 Hero를 변경한다.

조작권 전환과 이전 공격의 종료를 분리

일반 교대에서 이동·대기 상태는 SwapOut으로 전환하지만, 공격 중인 Hero는 오프필드 행동(HeroActionRegistry.IsOffFieldActionActive)으로 남아 동작을 마무리할 수 있습니다. 현재 Hero가 바뀌는 시점과 이전 Hero가 화면에서 사라지는 시점을 구분합니다.

PartyStateManager.cs
PartySwapManager.cs
HeroSwapSystem.cs

교대와 특수 교대

02 패링 지원 — 교대 요청과 실제 패링 성공을 분리

설계 배경

패링 가능 시간에 교대하더라도, 실제 공격이 새 Hero에게 도달했는지는 별도로 확인해야 합니다. 여러 적이 동시에 공격할 때는 지원을 요청한 공격과 다른 공격을 구분해야 합니다.

패링 지원의 실행 순서

01 CombatManager

OpenDefenseWindow
적의 공격 이벤트로 패링 가능 창을 열고 해당 공격자를 기록한다.

↓

02 PartySwapManager → PartyStateManager

타입 결정 · 교대 후보 조회
패링 가능 창이 열려 있으면 Parry 교대를 선택하고, 교대 가능한 Hero를 조회한다.

↓

03 PartySwapManager → CombatManager

TryConsumeParryAssistRequest
들어올 Hero와 공격자를 한 쌍으로 기록한다. 이 단계는 패링 성공 판정이 아니다.

↓

04 PartySwapManager → HeroSwapSystem · Hero

ExecuteSwap → ParryIn
이전 Hero의 퇴장을 처리하고 새 Hero를 배치한 뒤 패링 준비 상태로 진입시킨다.

↓

05 CombatManager → Hero

실제 적중 → TryResolveParryAssist
피격 Hero·공격자가 기록된 쌍과 일치하고 패링 반응이 가능할 때 성공 처리한다.

실제 공격을 확인한 뒤 지원 결과 적용

성공 시 공격자에게 패링 그로기 누적 처리를 적용하고 새 Hero의 패링 반응을 진행합니다. 이후 반격은 Hero의 반격 입력·예약과 애니메이션 이벤트에 따라 연결됩니다. 교대 입력만으로 패링과 반격이 모두 확정되지는 않습니다.

연속 패링 시 이전 상태 정리

HeroSwapSystem은 이전 Hero가 ParryIn이면 반격 예약과 대상을 정리하고 InActive로 전환합니다. 다른 공격 중이라면 오프필드 행동으로 남길 수 있어, 교대 직전 상태에 따라 퇴장 처리가 달라집니다.

CombatManager.cs
PartySwapManager.cs
HeroSwapSystem.cs

교대와 특수 교대

03 빠른 지원 — 발동 이벤트를 교대 요청과 연결

지원 창을 여는 조건

Hero가 피격으로 넉백 상태에 진입하거나 Erika·Eve의 지정된 공격 이벤트가 발생하면 QuickAssistManager.TryOpen을 호출합니다. 현재 Hero와 지원 후보를 확인한 뒤, 지원 Hero·공격 대상·위치·방향을 기록하고 제한 시간 동안 창을 엽니다.

빠른 지원의 실행 순서

01 Hero / ErikaMechanic / EveMechanic

QuickAssistManager.TryOpen
발동 조건을 만족하면 PartyStateManager에서 지정 방향의 생존 Hero를 지원 후보로 얻는다.

↓

02 PartySwapManager

TryRequestSwap → TryRequestQuickAssist
지원 창이 열려 있으면 일반 교대보다 먼저 빠른 지원 요청을 처리한다.

↓

03 QuickAssistManager

TryGetExecutionData
현재 Hero와 지원 후보를 다시 확인하고 QuickAssistContext로 실행 정보를 전달한다.

↓

04 PartySwapManager → HeroSwapSystem · Hero

ExecuteSwap → QuickAssist
지원 Hero의 기존 오프필드 행동을 정리하고, 대상 부근에 등장시켜 지원 공격 상태로 전환한다.

↓

05 PartySwapManager → QuickAssistManager

교대 실행 성공 → Complete
세션 ID와 Hero 쌍이 일치하면 지원 창을 소비한다. 공격 애니메이션 종료를 기다리지 않는다.

일반 교대와 다른 후보·대상 처리

일반 교대는 InActive 상태의 후보를 찾지만, 빠른 지원은 생존 후보를 구한 뒤 필요하다면 오프필드 행동을 정리합니다. 빠른 지원 교대 실행 시 대상이 유효하면 현재 위치·방향을 사용하고, 사망·비활성화 등으로 유효하지 않으면 지원 창을 열 때 저장한 위치·방향을 사용합니다.

지원 창의 수명 관리

새 요청은 기존 창을 교체하며, 시간 만료·유효하지 않은 상태·소비 완료 시 창을 닫습니다. 세션 ID와 Hero 쌍 검사는 현재 열린 요청에 해당하는 완료 처리인지 확인하는 역할을 합니다.

QuickAssistManager.cs
PartySwapManager.cs
PartyStateManager.cs

교대와 특수 교대

04 콤보스킬 — 후보 선택과 교대 공격을 단계별로 관리

콤보스킬을 여는 조건

현재 조작 Hero의 콤보스킬 유발 가능 타격이 그로기 상태의 적에게 적용되면 CombatManager가 ChainAttackManager.TryOpenChainSelection을 호출합니다. 남은 기회·선택 후보·진행 단계와 다른 연출 상태를 검사한 뒤 선택을 시작합니다.

콤보스킬의 실행 순서

01 ChainAttackManager → PartyStateManager

TryOpenChainSelection · 후보 구성

앞뒤 생존 Hero 중 현재 Hero와 이미 사용한 Hero를 제외한다. 선택을 열 때 그로기 기회를 소비한다.

↓

02 ChainAttackManager

Selecting · 전용 선택 입력

빠른 지원 창을 닫고 선택 입력과 제한 시간을 활성화한다. 선택 대기 시간은 unscaled time으로 관리한다.

↓

03 ChainAttackManager → PartySwapManager

선택 확정 → TryExecuteChainSwap

선택된 Hero를 검증하고 콤보스킬 전용 교대 경로로 퇴장·등장·현재 Hero 전환을 실행한다.

↓

04 HeroSwapSystem · Hero / ChainAttackManager

ChainAttack 진입 · Executing

Hero의 교대 처리를 거쳐 공격을 실행하고, 선택 입력을 해제하며 사용한 Hero를 기록한다.

↓

05 CombatManager / Hero → ChainAttackManager

추가 타격 → 다음 선택 / 공격 완료 → 세션 종료

조건을 만족한 추가 타격은 선택을 다시 연다. 현재 실행 Hero의 완료는 세션을 정리한다.

일반 교대와 실행 경로를 구분

일반·패링·빠른 지원은 ExecuteSwap을 공유하지만, 콤보스킬은 TryExecuteChainSwap을 사용합니다. ChainAttackManager가 선택·실행 단계를 관리하고 PartySwapManager가 실제 Hero 전환을 담당합니다.

그로기 종료와 공격 종료를 구분

Executing 중 그로기가 끝나도 진행 중인 모션은 유지합니다. 추가 선택은 다시 그로기·기회·후보 조건을 만족해야 합니다. 완료 처리도 단계와 현재 실행 Hero를 확인해 다른 Hero의 종료가 현재 세션을 정리하지 않도록 합니다.

ChainAttackManager.cs
PartySwapManager.cs
CombatManager.cs

캐릭터 기믹 확장

01 공통 구조 — Hero의 상태 전환에 캐릭터 전용 로직을 연결

공통 상태와 캐릭터 규칙의 책임 분리

Hero는 enum·switch 기반 FSM으로 현재 상태와 공통 이동·공격·피격·교대를 처리합니다.
같은 오브젝트의 IHeroMechanic 구현체를 조회해 전용 처리를 호출하며, Mechanic도 상태 변경이 필요하면 Hero 본체의 ChangeState로 요청합니다.

Hero.ChangeState의 실행 순서

01 Hero → IHeroMechanic
Mechanic.ExitState
기존 상태에 연결된 전용 동작·플래그를 먼저 정리한다.

↓

02 Hero
ExitState → 현재 상태 변경
공통 종료 처리를 실행한 뒤 _currentState를 새 HeroState로 변경한다.

↓

03 Hero
EnterState
새 상태에 필요한 공통 액션·이동·애니메이션 처리를 수행한다.

↓

04 Hero → IHeroMechanic
Mechanic.EnterState
공통 진입 처리 이후 캐릭터별 자원·전용 단계·추가 제약을 설정한다.

프레임 갱신과 전투 처리의 연결 지점

호출 지점	전용 기믹의 역할
Hero.Update	Hero.UpdateState 다음에 Mechanic.UpdateState를 호출한다.
애니메이션 이벤트	CanProcessAnimationEvent로 해당 전용 동작의 이벤트인지 검사한다.
공통 자원 UI	ResourceType·CurrentResource·MaxResource로 자원 표시를 조회한다.

신규 캐릭터 추가 시 확장 범위

공통 Hero와 전용 Mechanic을 연결하고 전용 상태 처리·이벤트·데이터를 구성하는 방식입니다. 다만 새 상위 행동이 필요하면 HeroState와 공통 분기도 확장해야 하며, Animator·전투 데이터·프리팹 연결까지 함께 검증해야 합니다.

Hero.cs
Interfaces.cs

캐릭터 기믹 확장

02 Maria — 에너지 소비를 전용 게이지와 강화 상태로 연결

설계 배경

Maria는 공통 공격 상태를 그대로 사용하면서 에너지 소비에 따라 강화가 유지되어야 합니다.
MariaMechanic은 이벤트 기반으로 에너지 소비량을 전용 게이지로 바꾸고,
게이지·생존 상태에 맞춰 자신이 등록한 Blazing 버프를 관리합니다.

게이지 획득과 강화 적용 흐름

01 HeroStatus → MariaMechanic

OnEnergyConsumed → HandleEnergyConsumed
실제 에너지 감소량을 알린다. 생존한 Maria의 Skill 상태에서 소비량에 비례해 게이지를 충전한다.

↓

02 MariaMechanic → HeroStatus

SynchronizeBlazingBuff
게이지가 남아 있으면 보관 중인 버프를 등록한다. 이미 적용된 버프를 중복 등록하지 않는다.

↓

03 Hero → MariaMechanic

UpdateState · 게이지 소모
현재 조작 Hero가 Maria일 때 Time.deltaTime 기준으로 게이지를 소모하고 버프 상태를 동기화한다.

↓

04 MariaMechanic

강화 종료 또는 사망 → 버프 해제
게이지가 소진되면 자신의 버프를 제거한다. 사망 진입 시 게이지와 버프를 함께 초기화한다.

공통 상태 전환과 연결되는 규칙

Ult 진입에서는 전용 게이지를 최대치로 충전합니다. 일반 교대 시에는 게이지를 초기화하지 않으며, 현재 조작 Hero가 아니면 시간에 따른 소모도 멈춥니다. 따라서 교대 후 다시 등장해 남은 강화를 이어갈 수 있습니다.

강화 수치와 표시의 동기화

버프는 공격력·치명타 확률·치명타 피해 효과를 묶어 HeroStatus에 적용합니다. 강화 상태가 실제로 달라질 때만 변경 이벤트를 알리고, 무기의 전투·비전투 표시 상태에 맞는 불꽃을 재생합니다. 강화 판정과 표시 갱신은 같은 활성 상태를 기준으로 연결됩니다.

MariaMechanic.cs · 게이지·강화 적용
HeroStatus.cs · 에너지 소비 알림
MariaMechanic.cs · 프레임 갱신

캐릭터 기믹 확장

03 Erika — 차지 공격 안에서 반복·자동 회피·마무리를 전환

상위 상태와 공격 내부 단계를 구분

Erika는 HeroState.ChargeAttack을 유지하면서 ChargeAttackPhase의 Loop·AutoDodge·Finisher로 동작을 구분합니다. 전용 단계와 애니메이션은 ErikaMechanic이 관리하고, 차지 공격 자체의 진입·종료는 Hero의 공통 FSM을 따릅니다.

자동 회피와 공격 복귀 흐름

01 Hero → ErikaMechanic

ChargeAttack 진입 → Loop

액션 무적을 설정하고 수동 회피·일반 교대 창을 막는다. 전용 자동 회피와 지정 이벤트의 빠른 지원을 사용한다.

↓

02 CombatManager → ErikaMechanic

TryEvadeHit → AutoDodge

Loop 재생 중 피격 판정이 들어오면 전용 회피를 시작한다. 입력 방향이 없으면 캐릭터 후방을 사용한다.

↓

03 ErikaMechanic → HeroMovement · Animator

회피 방향 설정 · 회피 모션 재생

콤보·캔슬 상태를 정리하고, 결정된 회피 방향으로 이동하도록 설정한다. 회피 중에는 공통 전투 회전을 제한한다.

↓

04 애니메이션 이벤트 → ErikaMechanic

OnChargeAttackAutoDodgeEnd

회피 방향의 이동 설정을 해제하고, 공격 입력 유지 시 Loop로 복귀한다. 입력을 놓았다면 Finisher로 전환한다.

↓

05 Hero → ErikaMechanic

ChargeAttack 종료 → 전용 상태 정리

수동 회피 차단·액션 무적을 해제하고 내부 단계를 None으로 초기화한다.

전환 이전 모션의 이벤트를 구분

CanProcessAnimationEvent는 이벤트의 Animator 상태 해시를 현재 ChargeAttackPhase와 비교합니다. 반복·회피·마무리 중 다른 단계에서 발생한 이벤트가 타격 인덱스나 종료 흐름을 바꾸지 않도록 연결 지점에서 걸러냅니다.

공격 성공을 전용 자원으로 연결

차지 공격의 성공 알림 OnAttackHitConfirmed를 받으면 자원을 획득합니다. 대상마다 자원을 더하는 대신 공통 타격 성공 알림 단위로 처리하며, 강화 특수 스킬의 지정 이벤트에서도 자원을 추가합니다.

ErikaMechanic.cs · 상태 진입·종료

ErikaMechanic.cs · 자동 회피

CombatManager.cs · 타격 성공 알림

캐릭터 기믹 확장

04 Erika — 오프필드에서 조건을 확인하고 여진 공격을 실행

여진 공격의 실행 조건

Erika가 InActive 상태일 때 생존·쿨타임·보유 자원을 확인합니다. 유효한 전투 대상과 안전한 등장 위치까지 확보한 경우에만 Hero.ChangeState(AfterShock)를 요청합니다. 조작 Hero를 바꾸지 않고 전용 공격을 실행합니다.

등장부터 자원 소비·퇴장까지

01 Hero → ErikaMechanic
UpdateState → TryBeginAftershock
대기 중 조건을 검사하고 MagneticTarget을 이번 여진의 고정 대상으로 저장한다.

↓

02 ErikaMechanic
TryFindAfterShockStartPosition
후보 위치의 바닥·경사·캡슐 점유·대상과의 벽 차단을 검사한다. 위치를 찾지 못하면 진입하지 않는다.

↓

03 ErikaMechanic → Hero → HeroSwapSystem
AfterShock 진입 · 등장 준비
확보한 위치를 적용하고 오프필드 행동을 활성화한다. 표시·물리를 준비한 뒤 여진 모션을 재생한다.

↓

04 애니메이션 이벤트 → ErikaMechanic → HeroHitChecker
첫 타격에서 대상 확인 → 자원 소비 → 판정
첫 실행 프레임에 대상을 다시 검사하고 자원을 소비한다. 이후 타격은 공통 여진 판정으로 전달한다.

↓

05 ErikaMechanic → Hero
OnAfterShockEnd → InActive
종료 시 공통 대기 상태로 돌아간다. 전용 ExitState에서 대상·트리거·오프필드 행동을 정리한다.

자원을 소비하는 시점을 분리

등장 위치를 확보하지 못하면 상태·쿨타임·자원을 변경하지 않습니다. 진입한 뒤 첫 타격 전에 대상이 무효해지면 자원을 소비하지 않고 종료합니다. 진입 검증과 실제 공격 비용 지불을 나눕니다.

여진 공격 중 조준 대상 유지

Erika는 여진 시작 시 공격 대상을 저장합니다. ProjectileLauncher는 여진 투사체를 발사할 때 현재 록온 대상 대신 IAfterShockTargetProvider에서 저장된 대상을 조회해 발사 방향을 계산합니다. 이를 통해 조작 중인 Hero의 록온 대상이 바뀌어도 Erika는 처음 지정한 적을 향해 여진 공격을 이어갑니다.

※ 현재 이 대상 선택 규칙은 투사체 발사 경로에만 구현되어 있습니다. 근접·설치형 공격에도 같은 조준 규칙이 필요해지면, 대상 선택을 각 판정 실행기와 분리하는 구조를 검토해야 합니다.

ErikaMechanic.cs · 여진 진입 조건
ErikaMechanic.cs · 첫 타격·자원 소비
Interfaces.cs · IAfterShockTargetProvider

적 AI

01 전체 구조 — 감지·판단·행동 선택·상태 실행의 책임 분리

각 요소가 담당하는 역할

구성 요소	역할 및 책임
EnemySensor	대상 존재 여부와 거리를 읽어 Blackboard에 기록한다.
EnemyBlackboard	대상·거리·이동 범위·선택 패턴·다음 대기 행동을 공유한다.
EnemyDecision	거리 조건을 판단하고 Pattern에 공격 패턴 선택을 요청한다.
EnemyPattern	사거리·쿨타임·가중치·사용 이력으로 패턴을 선택한다.
BT	판단 결과를 읽어 공격·대기·주변 이동·추격의 우선순위를 평가한다.
Enemy FSM	선택된 행동의 이동·애니메이션·중단·종료를 처리한다.

EnemyAI.Update의 실행 순서

01 EnemyAI → EnemySensor
Sensor.Tick · 관측 정보 갱신
Enemy가 제공하는 HeroTransform을 읽고 대상·거리 정보를 Blackboard에 기록한다.

↓

02 EnemyAI → EnemyDecision → EnemyPattern
Decision.Tick · 이동 조건과 공격 후보 갱신
거리 판단과 패턴 선택 결과를 Blackboard의 범위 플래그·SelectedAttackPattern·CanAttack에 반영한다.

↓

03 EnemyAI → BT
실행 허용 확인 → 루트 Evaluate
AI 실행 중단 여부를 검사한 뒤 Normal 모드에서 행동 트리를 평가한다.

↓

04 BT → Enemy
ExecuteAttack / ChangeState
선택한 행동을 FSM에 요청한다. 실제 이동·모션·타격은 Enemy와 관련 컴포넌트가 수행한다.

Blackboard와 실행 중 패턴을 구분

Blackboard는 판단에 쓰는 공유 정보이며, 실행 중 공격은 Enemy의 _currentAttackPattern에 보관합니다. 판단 후보가 갱신되어도 이미 시작한 공격의 패턴을 그대로 교체하지 않습니다.

EnemyAI.cs
EnemyBT.cs
Enemy.cs

적 AI

02 Sensor · Blackboard · Decision — 관측 정보에서 행동 조건으로

대상과 거리를 갱신하는 흐름

01 Enemy

현재 파티 Hero → HeroTransform

PartyStateManager의 현재 Hero를 공격 대상 참조로 갱신한다.

↓

02 EnemySensor → EnemyBlackboard

Target · HasTarget · DistanceToTarget

대상 참조와 두 오브젝트 사이의 거리를 기록한다. 대상이 없으면 거리를 Infinity로 설정한다.

↓

03 EnemyDecision → EnemyBlackboard

추격·주변 이동 범위 판단

추격과 Orbit 각각의 진입·이탈 기준으로 IsInChaseRange와 IsInOrbitDistance를 갱신한다.

↓

04 EnemyDecision → EnemyPattern → EnemyBlackboard

GetAttackPattern → 공격 가능 여부

Normal 모드에서 선택한 패턴을 저장한다. 결과가 null이면 CanAttack은 false가 된다.

거리 경계에서 판단이 반복되는 것을 줄이기

Decision은 진입 거리와 이탈 거리를 따로 둡니다. 범위 밖에서는 진입 거리 이내일 때 활성화하고, 이미 범위 안이라면 더 먼 이탈 거리에 도달할 때 해제합니다. 두 기준 사이에서는 이전 판단을 유지하는 히스테리시스 방식입니다.

판단 정보 갱신과 행동 실행을 구분

EnemyAI는 사망 시 갱신을 종료합니다. 살아 있는 적은 공격·피격 등으로 BT 실행이 중단되어도 Sensor와 Decision을 먼저 갱신합니다. 이때 새 후보를 계산할 수 있지만, BT가 실행되지 않으므로 새 행동 요청으로 이어지지는 않습니다.

현재 구현의 감지 범위

Sensor는 Enemy가 전달한 현재 Hero와 거리를 읽는 역할입니다. 시야각·가림막·소리로 대상을 탐색하는 기능은 포함하지 않습니다. 또한 Enemy.Update와 EnemyAI.Update의 상호 실행 순서는 이 코드만으로 보장되지 않으므로, 같은 프레임의 최신 대상 반영까지 단정하지 않습니다.

EnemySensor.cs

EnemyDecision.cs

EnemyAI.cs

적 AI

03 Pattern — 실행 가능한 후보를 걸러 가중치로 선택

설정 데이터와 개체별 사용 기록 분리

EnemyPatternDataSO에는 사거리·쿨타임·기본 선호도·거리 곡선·애니메이션 트리거·CombatData를 둡니다. EnemyPattern은 이를 EnemyAttackPattern으로 감싸 적마다 마지막 종료 시각과 사용 이력을 따로 관리합니다.

GetAttackPattern의 선택 흐름

01 EnemyPattern
전역 쿨타임 확인
패턴 사이의 공통 대기 시간이 남아 있으면 후보를 선택하지 않는다.

↓

02 EnemyAttackPattern
사거리·개별 쿨타임 검사
현재 거리가 최소~최대 사거리 안에 있고, 해당 패턴의 쿨타임이 끝난 경우만 후보로 남긴다.

↓

03 EnemyAttackPattern / EnemyPattern
거리 적합도·반복 이력으로 점수 계산
기본 선호도에 거리 곡선 값과 반복 보정 배율을 곱한다. 최종 점수가 0 이하인 후보는 제외한다.

↓

04 EnemyPattern → EnemyDecision
가중치 추첨 → 선택 결과 반환
양수 가중치의 합에서 난수를 뽑아 후보 하나를 선택한다. 후보가 없으면 null을 반환한다.

점수는 확률 가중치로 사용

가중치 = 기본 선호도 × 거리 적합도 × 직전 반복 배율 × 최근 이력 배율
가장 높은 점수를 항상 선택하는 방식은 아닙니다. 최근 종료한 패턴은 이력 순서에 따라 감점되지만, 가중치가 남아 있으면 다시 선택될 수 있습니다.

선택 조회와 사용 이력 갱신을 분리

GetAttackPattern은 후보를 조회할 뿐 쿨타임이나 사용 이력을 갱신하지 않습니다. Enemy가 Attack 상태를 벗어날 때 MarkPatternEnded를 호출하며, 정상 완료와 피격·그로기 등으로 중단된 경우 모두 개별·전역 쿨타임과 최근 이력에 반영합니다.

검증해야 할 부분

원하는 선택 빈도는 가중치 설정만으로 보장되지 않습니다. 거리·후보·최종 가중치·선택 결과를 기록해 분포를 확인해야 하며, 공격 중에도 수행되는 패턴 재평가의 비용은 별도 측정 대상입니다.

EnemyPattern.cs
EnemyPatternDataSO.cs
EnemyPattern.cs · 종료 이력 기록

적 AI

04 BT · Enemy FSM — 행동 우선순위와 실행 수명 관리

현재 행동 트리의 평가 우선순위

우선순위	조건 → 행동
1 공격	CanAttack → 선택한 패턴으로 ExecuteAttack
2 전투 대기	Orbit 범위 + 전역 쿨타임 + 대기 차례 → CombatIdle
3 주변 이동	Orbit 범위 + 전역 쿨타임 + 이동 차례 → Orbit
4 추격 / 5 대기	대상 존재 + 추격 범위 → Chase / 앞선 조건 실패 → Idle

MutantAI와 BerserkerAI는 현재 같은 우선순위로 트리를 구성합니다. Selector는 먼저 성공하거나 실행 중인 분기를 선택하고, Sequence는 조건부터 순서대로 평가합니다. 매 평가마다 앞에서 다시 검사하며 자식 실행 인덱스를 저장하지 않습니다.

공격 선택부터 완료·중단까지

01 BTAction_Attack → Enemy
ExecuteAttack → Attack 진입
선택 패턴을 저장하고 BT 실행을 중단한다. 패턴 데이터에 맞춰 슈퍼아머와 공격 애니메이션을 설정한다.

↓

02 애니메이션 이벤트 → EnemyHitChecker
OnAttackExecuteFrame → 타격 판정
모션·히트 인덱스를 읽어 실제 타격을 요청한다. BT는 타격 프레임이나 피해 계산을 수행하지 않는다.

↓

03 Enemy FSM → EnemyPattern
Attack 종료 → MarkPatternEnded
정상 종료와 중단 모두 회전 추적·방어 창·슈퍼아머 등을 정리하고 종료 이력을 기록한다.

↓

04 Enemy FSM → EnemyAI
Idle 복귀 → BT 실행 재개
정상 공격 종료는 Idle로 돌아간다. 피격·그로기로 중단되면 해당 상태가 끝날 때까지 행동 실행을 보류한다.

Running과 애니메이션 완료는 별개

공격 노드는 ExecuteAttack 요청 후 Success를 반환합니다. 공격 모션의 진행·종료는 FSM과 애니메이션 이벤트가 관리합니다. Chase·Orbit의 Running도 다음 평가를 막지 않으며, Orbit 노드는 목적지 도착 또는 제한 시간에 따라 대기 차례를 전환합니다.

BerserkerAI.cs
EnemyBT.cs
Enemy.cs

교대 퇴장 중 이동 오류 해결

충돌 제어와 CharacterController 비활성화 시점을 분리

문제와 원인 추적

교대 퇴장 모션에서 HeroMovement의 이동 처리가 의도대로 이어지지 않았습니다. SwapOut 진입은 충돌을 끄기 위해 SwitchHeroCollider(false)를 호출했지만, 기존 구현은 GetComponent<Collider>()로 얻은 컴포넌트의 enabled를 껐습니다.

이 경로에서 CharacterController까지 비활성화되면 ExecuteCombatAnimMovement가 enabled 검사에서 반환합니다. 퇴장 방향·속도 계산보다 앞에서 이동이 차단된 것이 핵심 원인이었습니다.

수정 전후의 책임 분리

수정 전	수정 후
Collider.enabled = false	퇴장 중에는 CharacterController.detectCollisions만 변경하고 컨트롤러 활성 상태는 유지한다.
퇴장 종료와 비활성화 연결	퇴장 이동·디더를 먼저 마친 뒤 InActive 진입에서 컨트롤러를 비활성화한다.

수정 후 퇴장 처리 순서

01 Hero → HeroSwapSystem
SwapOut 진입 · 충돌 감지 제어
컨트롤러를 활성 상태로 유지해 퇴장 이동 경로가 실행될 수 있도록 한다.

↓

02 애니메이션 이벤트 → HeroSwapSystem
OnSwapOutFadeStart → BeginSwapOutFade
퇴장 상태를 유지한 채 디더를 진행한다. 이동·모션이 끝나기 전에 InActive로 전환하지 않는다.

↓

03 디더 완료 콜백 → Hero
CompleteSwapOut → InActive
완전히 숨긴 뒤 오프필드 행동을 해제하고, InActive 진입에서 표시·컨트롤러를 정리한다.

결과와 배운 점

충돌 제어, 이동 가능 여부, 화면 표시 종료를 서로 다른 시점에 처리하도록 수정했습니다. 완료 콜백에서도 SwapOut 상태인지 검사해 상태가 바뀐 뒤 늦게 도착한 완료가 다른 행동을 종료하지 않도록 했습니다.

CharacterController는 Collider를 상속해 GetComponent<Collider>()로도 조회될 수 있습니다. enabled를 끄면 이동 컴포넌트까지 비활성화됩니다. 반면 detectCollisions는 다른 물체의 접근을 막는지를 제어하며, 자신의 이동 중 장애물 검사는 유지합니다. 이를 통해 컴포넌트의 상속 관계를 확인하고, 활성 상태와 충돌 제어를 구분해야 함을 배웠습니다.

HeroSwapSystem.cs
HeroAnimationEventHandler.cs
HeroDitherController.cs

궁극기 프리팹 사전 풀링

최초 실행 시 필요한 객체 생성 작업을 준비 단계로 이동

개선 대상과 기존 생성 경로

무거운 궁극기 프리팹을 처음 사용할 때 필요한 객체 생성이 전투 실행 시점에 몰리는 경로에 주목했습니다. 기존 ParticleVFXManager는 빈 풀에서 재생을 요청하면 Instantiate했고, 설치형 풀도 최초 생성 시 인스턴스를 준비하는 구조였습니다.

반환된 객체를 재사용하는 것만으로는 첫 실행의 생성 비용을 피할 수 없습니다. 현재 파티가 사용하는 궁극기 데이터를 읽어, 해당 프리팹의 객체를 미리 준비하는 단계를 추가했습니다.

사전 준비부터 재사용까지

01 풀 매니저 → PartyStateManager
Start · OnPartyMembersChanged
초기화와 파티 변경 시 현재 멤버의 CombatData.ultAttackSequences를 순회한다.

↓

02 ParticleVFXManager / PlacedAreaPoolManager
궁극기 프리팹별 풀 준비
근접 판정의 hitVFXPrefab과 설치형 placedAreaPrefab을 각각 확인해 설정 수량만큼 사전 생성한다.

↓

03 궁극기 실행 → 풀 매니저
대기 객체 대여 · 부족할 때 추가 생성
이미 준비된 객체를 대여해 연출·공격을 실행한다. 풀에 객체가 없으면 생성하는 경로는 유지한다.

↓

04 연출·공격 종료 → 풀 매니저
비활성화·상태 초기화 → 반환
근접 연출은 비활성화하고, 설치형은 ResetForPool로 실행 정보를 정리한 뒤 프리팹별 대기열에 돌려준다.

범위와 수량을 제한한 이유

현재 파티의 궁극기에 참조된 프리팹을 대상으로 준비하며, 이미 등록된 풀은 다시 만들지 않습니다. 궁극기 사전 생성 수량을 별도로 두되, 기존 풀의 대기 수량을 매번 보충하는 기능은 아닙니다. 준비 객체 수만큼 메모리 점유와 초기화 비용이 늘어납니다.

결과와 배운 점

현재 파티의 궁극기 프리팹을 사전 생성하도록 변경해, 준비된 수량 안에서는 실행 시 새 객체를 생성하지 않고 풀에서 대여하도록 했습니다. 준비 수량을 초과하면 추가 생성하며, 해당 객체도 종료 후 풀에 반환해 재사용합니다.

이를 통해 객체 풀링은 재사용뿐 아니라 언제, 무엇을, 얼마나 미리 생성할지까지 고려해야 한다는 점을 배웠습니다. 사전 생성은 실행 시점의 생성 부담을 줄이는 대신 초기화 비용과 메모리 점유를 늘리므로, 사용 빈도와 동시 실행량에 맞춰 준비 대상을 선정하고 수량을 조절해야 합니다.

ParticleVFXManager.cs
PlacedAreaPoolManager.cs
HeroCombatDataSO.cs

핵심 역량과 향후 발전 방향

전투 시스템 구현에서 상태 관리·확장 설계·문제 해결까지

프로젝트로 보여주는 핵심 역량

핵심 역량	구현 근거
공통 처리와 책임 분리 04p ~ 05p	근접·투사체·설치형 판정을 공통 피해 처리로 연결하고, CombatManager의 계산과 Hero·Enemy의 피해 적용을 분리했습니다.
상태와 실행 수명 관리 06p ~ 09p	일반 교대·패링 지원·빠른 지원·콤보스킬의 진입 조건과 실행 흐름을 구성하고, 조작권 전환과 잔여 행동을 구분했습니다.
캐릭터별 기능 확장 10p ~ 13p	Hero FSM과 IHeroMechanic을 연결해 Maria의 강화, Erika의 자동 회피·여진을 공통 상태 흐름 안에서 구현했습니다.
AI 판단과 행동 실행 14p ~ 17p	Sensor·Blackboard·Decision·Pattern의 판단 결과를 BT와 Enemy FSM으로 연결하고, 패턴 선택과 실행 종료를 구분했습니다.
원인 분석과 개선 18p ~ 19p	퇴장 이동 오류의 컴포넌트 비활성화 경로를 수정하고, 궁극기 객체 생성 시점을 사전 준비 단계로 옮겼습니다.

프로젝트를 통해 배운 개발 방식

기능을 추가하는 것뿐 아니라 데이터가 어디서 만들어지고, 누가 상태를 바꾸며, 실행이 언제 끝나는지를 함께 설계했습니다. 문제가 발생하면 실제 호출 순서와 컴포넌트 상태를 추적하고, 원인을 수정한 뒤 연관된 동작에도 영향이 없는지 확인하는 습관을 길렀습니다.

다음 발전 방향

성능을 측정하고 준비 수량 조정

첫 궁극기와 반복 사용의 CPU 시간·Instantiate 호출·메모리를 비교하고, 동시 실행량과 풀 고갈 기록을 기준으로 사전 생성 수량을 조정하겠습니다.

공격 대상 선택 규칙의 공통화 검토

여진의 고정 대상 조준을 근접·설치형으로 확장할 필요가 생기면, 대상·방향 결정과 각 판정 실행을 분리해 같은 규칙이 여러 실행기에 중복되지 않도록 하겠습니다.

전투 기록 기반 플레이 분석 및 디버깅 지원

공격 적중, 피해량, 교대 및 지원 발동 등의 전투 이벤트를 기록하는 분석 기능을 고려하고 있습니다. 사용자에게는 전투 결과를 바탕으로 자신의 플레이를 돌아볼 수 있는 피드백을 제공하고, 개발자에게는 이벤트 발생 순서와 당시 상태를 확인할 수 있는 로그를 제공해 문제 원인을 추적하기 쉽게 만들고자 합니다.

협업 경험과 소통 역량 확장

현재까지는 1인 개발로 프로젝트를 진행해 팀 단위의 개발 경험은 없습니다. 앞으로 공동 프로젝트에 참여해 자신의 구현 의도를 설명하고 다른 직군의 의견을 듣는 경험을 쌓고 싶습니다. 특히 역할을 나누고, 서로의 작업물에 대한 피드백을 주고받으며, 함께 정한 기준에 맞춰 개발하는 방식을 배우고자 합니다.

GitHub / 소스코드와 README
플레이 영상 링크
실행 파일 링크